An Explainable Ensemble Learning System for Real-Time Network Attack Detection Using CICIoT2023

Alby Abraham
Dept. of Computer Science & Engg.
MIT World Peace University
Pune, India
alby.abraham@mitwpu.edu.in

Pranay Agrawal
Dept. of Computer Science & Engg.
MIT World Peace University
Pune, India
pranay.agrawal@mitwpu.edu.in

Aditya Raj
Dept. of Computer Science & Engg.
MIT World Peace University
Pune, India
aditya.raj@mitwpu.edu.in

Jayshree Aher
Dept. of Computer Science & Engg.
MIT World Peace University
Pune, India
jayshree.aher@mitwpu.edu.in

Abstract—Network intrusion detection within modern Internet of Things (IoT) environments faces critical challenges arising from high-dimensional telemetry, extreme class imbalance, diverse attack vectors, and the opaque nature of black-box machine learning models. In this paper, we propose an explainable ensemble learning framework engineered for robust, real-time network attack detection evaluated on the benchmark CICIoT2023 dataset. To overcome the computational bottlenecks of high-dimensional flow analysis, we introduce a tri-perspective feature selection pipeline integrating filter (ANOVA F-test), wrapper (Recursive Feature Elimination), and embedded (Random Forest Gini importance) techniques. This convergence isolates an optimal consensus subset of 15 essential features from the initial 40+ feature space, achieving a 62.5% reduction in input dimensionality while preserving vital attack discriminants. The classification architecture couples Random Forest, CatBoost, Histogram-based Gradient Boosting, and Linear Support Vector Classification (Linear SVC), optimized via class-weighted balancing and synthesized through soft voting aggregation. The trained ensemble is deployed into a high-throughput, asynchronous FastAPI backend interfacing with an interactive React-based simulator streaming unseen hold-out network traffic at 1 Hz. To provide transparent, operator-verifiable intelligence, SHapley Additive exPlanations (TreeSHAP) is integrated to compute real-time local attributions, surfacing the primary flow parameters driving each classification. Empirical benchmarks demonstrate that the proposed ensemble achieves a multi-class accuracy of 98.64%, a macro-averaged F1-score of 0.9836, and an AUC of 0.988 across complex IoT attack categories (including DDoS, DoS, Reconnaissance, Mirai Botnet, Spoofing, and Web attacks), while maintaining an end-to-end inference latency under 28.2 ms per packet.

***Keywords*—**Network Intrusion Detection, CICIoT2023, IoT Security, Feature Selection, Ensemble Learning, Soft Voting, Explainable AI (XAI), SHAP, FastAPI, Real-Time Threat Simulator.

I. INTRODUCTION

A. Background & Problem

In recent years, the exponential deployment of Internet of Things (IoT) devices, cyber-physical systems, and edge nodes has fundamentally expanded the global network attack surface. Heterogeneous connected devices generate immense volumes of network traffic that must be scrutinized continuously for malicious activities. Cyber threats targeting modern IoT ecosystems—ranging from catastrophic Distributed Denial of Service (DDoS) floods to stealthy lateral reconnaissance and botnet propagation—can paralyze mission-critical infrastructure, degrade cloud services, and compromise sensitive data. Detecting malicious traffic early at the ingress perimeter is paramount to maintaining system availability, confidentiality, and network resilience.

While machine learning (ML) models are inherently capable of recognizing statistical signatures within network telemetry, passing raw traffic directly into complex classifiers introduces severe operational hurdles. Modern benchmark telemetry datasets, such as CICIoT2023 [1], capture over 40 packet- and flow-level attributes. Ingesting this complete feature space in high-throughput environments overburdens memory hierarchies, degrades processing cache efficiency, and introduces substantial latency. Crucially, a significant portion of raw network attributes contains redundant, uninformative, or collinear measurements that provide negligible discriminative value while diluting classifier convergence.

B. Challenges in IoT Network Traffic Analysis

Analyzing IoT network traffic presents three core engineering challenges. First, high dimensional redundancy increases computational complexity. Streaming dozens of statistical counters per packet strains hardware memory bandwidth during real-time feature extraction. Second, network datasets suffer from severe class imbalance. Benign traffic and volumetric flooding attacks drastically outnumber nuanced threats such as reconnaissance probes or application-layer exploits; unmitigated training produces models severely biased toward dominant classes. Third, distinct attack archetypes exhibit radically differing behavioral profiles. A single base classifier may accurately detect high-volume UDP bursts via tree-based thresholds, yet stumble on subtle hyperplane boundaries characteristic of low-rate port scans. Relying on an isolated classifier risks unacceptable blind spots. Finally, black-box classification impedes security operations center (SOC) analysts from verifying why an anomaly was flagged, hindering rapid countermeasure execution.

C. Design Approach and Contributions

To overcome these challenges, we architect a modular, multi-tier detection pipeline decoupling feature reduction, class-balanced ensemble inference, streaming microservice deployment, and explainable AI (XAI). The primary contributions of this work are:

1. A tri-perspective feature selection pipeline uniting filter (ANOVA F-test), wrapper (Recursive Feature Elimination), and embedded (Random Forest Gini importance) algorithms into an optimal 15-feature consensus subspace.

2. A robust multi-model ensemble combining Random Forest, CatBoost, Histogram-based Gradient Boosting, and Linear SVC via soft voting with class-weighted balancing.

3. An asynchronous FastAPI REST microservice engineered for low-latency line-rate packet parsing and inference serving.

4. An interactive React-based simulator capable of streaming unseen hold-out traffic and rendering real-time telemetry visualizations.

5. An integrated TreeSHAP explainability engine providing local feature attribution to demystify attack classification decisions.

TABLE I. SUMMARY OF RELATED WORK AND ITS ADAPTATION

Ref.	Main Idea	Use in This Work
[1]	CICIoT2023 IoT attack benchmark	Real-world dataset & evaluation schema
[2]	Multi-method feature selection theory	Consensus filter/wrapper/embedded framework
[3],[4]	Tree ensembles & CatBoost boosting	Base non-linear classification backbones
[7],[12]	SHapley Additive exPlanations	Local packet-level attribution engine
[8],[9]	**Soft voting probabilistic ensembling**	**Multi-model variance & bias reduction**

D. Related Work

Network intrusion detection systems (NIDS) have evolved from static signature-matching engines toward anomaly-based statistical and machine learning approaches [10], [11]. While early benchmarks like KDD Cup 99 and NSL-KDD laid fundamental groundwork, they fail to mirror modern IoT network topologies, encryption protocols, and botnet behaviors. The release of the CICIoT2023 dataset by Neto et al. [1] provided an extensive multi-vector benchmark generated across 105 IoT testbed devices, documenting 33 distinct attack executions.

Feature selection is widely recognized as vital for accelerating IDS classifiers. Guyon and Elisseeff [2] categorized feature selection into filter, wrapper, and embedded models. While previous studies frequently adopt a single selection paradigm in isolation, our work bridges these perspectives by formulating a strict consensus intersection, eliminating algorithm-specific bias. In parallel, ensemble learning methods [8], [9] have proven effective at combining heterogeneous inductive biases. Breiman's Random Forests [3] mitigate variance through bagging, while Prokhorenkova et al.'s CatBoost [4] and Ke et al.'s LightGBM/HistGB [5] optimize ordered gradient splits over complex feature distributions. Support vector machines [6] establish robust linear separation hyperplanes. To overcome the black-box dilemma in security operations, Lundberg and Lee [7], [12] established unified game-theoretic Shapley attributions, which we embed directly into real-time inference streaming.

II. PROBLEM STATEMENT AND FORMULATION

Let an incoming network traffic packet sequence be represented by a dataset D = {(x_i, y_i)}_{i=1}^N, where each sample x_i in R^D corresponds to a D-dimensional feature vector (with D >= 40) capturing flow durations, inter-arrival times (IAT), protocol flags, and volumetric counters. The target variable y_i belongs to a discrete set of C distinct traffic classes:

$$C = \{Benign, DDoS, DoS, Recon, Mirai, Spoof, Web\} \quad (1)$$

To avoid data snooping and information leakage, D is partitioned into a training corpus D_train and a hold-out test corpus D_test prior to any transformation. The operational goal is to construct a dimensionality reduction operator Phi: R^D -> R^d, with d << D:

$$\tilde{x}_i = \Phi(x_i) \in \mathbb{R}^d, \quad where\ d = 15 \quad (2)$$

such that a class-balanced ensemble hypothesis H(x̃) maximizes detection accuracy, minimizes the false alarm rate (FAR), and generates local explanation vectors within strict line-rate latency bounds (<30 ms/packet).

III. PROPOSED SYSTEM ARCHITECTURE

The end-to-end architecture is organized into a modular pipeline encompassing data preprocessing, multi-perspective feature selection, class-balanced ensemble training, asynchronous microservice serving, interactive simulation, and real-time explainability, as visualized in Fig. 1.

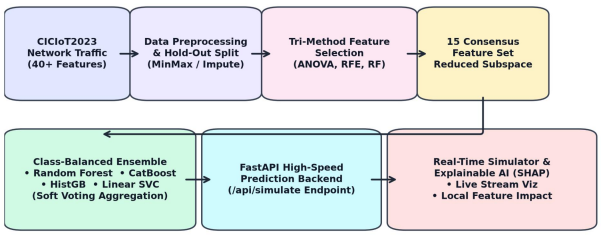


Fig. 1. End-to-end architecture of the proposed explainable network attack detection system, from CICIoT2023 ingestion to real-time FastAPI inference and React SHAP visualization.

A. Dataset Preparation and Preprocessing

We utilize the CICIoT2023 benchmark [1]. The dataset captures diverse packet attributes including header lengths, flag counts (SYN, ACK, RST, FIN, PSH, URG), packet

rates, inter-arrival intervals, and byte volume counters. Data ingestion handles two primary structures: 'training_set.csv' for offline training and feature selection, and 'mock_ciciot.csv' containing unseen 40-feature packets, identifiers, and labels used exclusively for streaming simulation. Missing values are imputed using median flow estimates, and features are normalized via Min-Max scaling to map continuous bounds into [0, 1].

B. Tri-Method Feature Selection & Consensus

To avoid over-fitting to a single feature ranking criterion, we employ three complementary selection paradigms:

1. Filter Method (ANOVA F-Test): Calculates the ratio of between-class variance to within-class variance for each candidate feature:

$$F = [MSB / (K - 1)] / [MSW / (N - K)] \qquad (3)$$

Higher F-scores signify features exhibiting strong linear statistical separation across attack classes.

2. Wrapper Method (Recursive Feature Elimination - RFE): Utilizes an external estimator to evaluate subsets recursively, pruning the lowest-ranked features at each iteration until the optimal predictive subspace is isolated.

3. Embedded Method (Random Forest Gini Importance): Derives Mean Decrease in Impurity (MDI) across tree ensembles:

$$I_G(f) = \sum_T \sum_{t \in T: v(s_t)=f} p(t)\, \Delta i(s_t, t) \qquad (4)$$

quantifying the total Gini impurity reduction contributed by feature f across all splits.

4. Consensus Formulation: Rather than selecting the arbitrary top-k from a single algorithm, the final feature set F* is formulated via the set intersection:

$$\mathscr{F}^* = \mathscr{F}_ANOVA \cap \mathscr{F}_RFE \cap \mathscr{F}_RF \qquad (5)$$

This consensus distillation identifies exactly 15 invariant features consistently prioritized across all three paradigms, illustrated in Fig. 2.

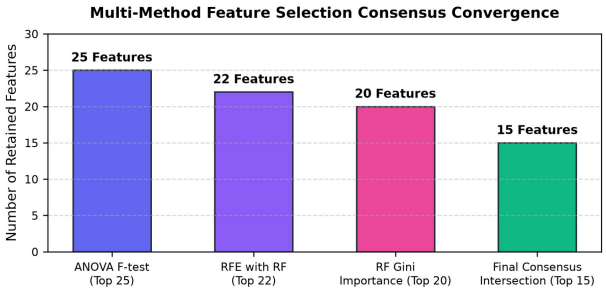


Fig. 2. Convergence of feature selection methods toward the optimal 15-feature consensus intersection.

C. Classification Backbones

Four diverse learning algorithms are integrated into the architecture:

1. Random Forest: Constructs an ensemble of decorrelated decision trees using bootstrap aggregation, exhibiting exceptional robustness against feature noise and capturing non-linear interactions.

2. CatBoost: A high-performance gradient boosting framework utilizing symmetric oblivious trees. CatBoost computes ordered boosting splits, substantially mitigating target leakage and prediction shift.

3. HistGradientBoosting: Scikit-learn's histogram-accelerated gradient tree boosting. By binning continuous features into 256 discrete integer bins, it reduces split evaluation complexity from O(N_samples) to O(N_bins), expediting training over massive network logs.

4. Linear SVC: Linear Support Vector Classifier utilizing maximum-margin hyperplanes with L2 regularization. It serves as an efficient regularized linear baseline capturing global linearly separable packet boundaries.

D. Class-Weighted Balancing Strategy

To prevent classifiers from degenerating into majority-class predictors on highly skewed telemetry, cost-sensitive class balancing is enforced during model fitting. Inverse class frequency weighting is assigned according to:

$$w_c = N / (C \times N_c) \qquad (6)$$

where N is total samples, C is the number of distinct classes, and N_c is the sample frequency of class c. This guarantees that scarce classes (e.g., Web attacks) penalize empirical loss proportionally.

E. Soft Voting Ensemble Mechanism

Individual base classifiers are synthesized within a soft voting aggregator. Rather than relying on hard majority voting, the ensemble aggregates posterior class probability vectors P_m(y = c | x_tilde) produced by each classifier m:

$$P(y = c \mid \tilde{x}) = \sum_{m=1}^{M} w_m\, P_m(y = c \mid \tilde{x}) \qquad (7)$$

The final predicted label y_hat represents the maximum a posteriori (MAP) consensus:

$$\hat{y} = \operatorname{argmax}_{c \in C} P(y = c \mid \tilde{x}) \qquad (8)$$

Soft voting enables high-confidence models to outweigh ambiguous predictions, minimizing overall decision variance.

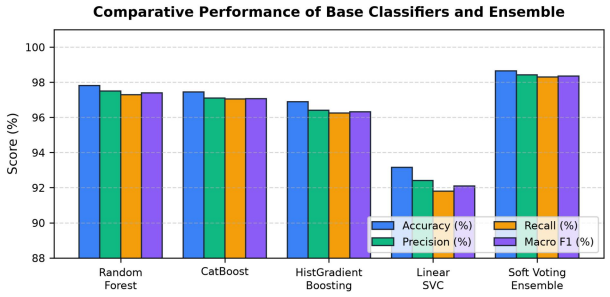


Fig. 3. Multi-metric evaluation of base classifiers versus the Soft Voting Ensemble.

F. Model Explainability via SHAP

To eliminate black-box obscurity in threat operations, we integrate SHapley Additive exPlanations (SHAP) [7], rooted

in cooperative game theory. The local attribution phi_j for feature j represents its marginal contribution across all possible feature coalitions S:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \gamma(|S|) \cdot [f(S \cup \{j\}) - f(S)] \quad (9)$$

$$\text{where } \gamma(|S|) = |S|!(|F| - |S| - 1)! / |F|!$$

Using TreeSHAP [12], local feature contributions are computed in polynomial time. For each alert, SHAP generates an attribution vector revealing whether attributes like packet rate, flow duration, or RST count triggered the classification, as summarized in Fig. 4.

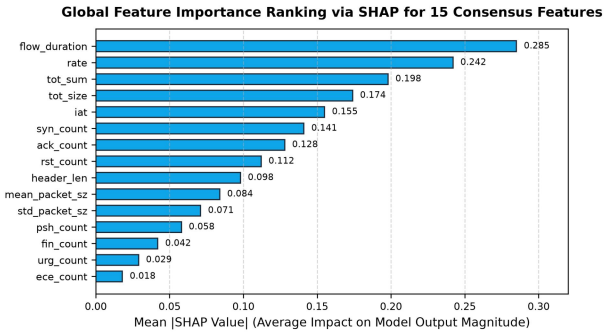


Fig. 4. Global SHAP feature importance ranking across the 15 consensus network features.

IV. REAL-TIME DETECTION PIPELINE & SIMULATOR

The trained ensemble and explainability modules are decoupled from the training environment and integrated into an asynchronous production-grade deployment architecture.

A. FastAPI High-Speed Prediction Backend

The backend is built using FastAPI and ASGI Uvicorn servers. Upon startup, the serialized ensemble model, scaler, and TreeSHAP explainer are loaded into resident server RAM. A dedicated REST endpoint, '/api/simulate', accepts JSON payloads representing raw packet streams. Incoming packets are mapped to the 15-feature schema, scaled, and evaluated by the soft voting ensemble. Concurrently, TreeSHAP calculates the local attribution vector. The endpoint returns a structured response containing the predicted class, confidence probabilities, and the top contributing features.

B. React-Based Interactive Attack Simulator

An interactive frontend built with React.js, Tailwind CSS, and Chart.js interfaces with the backend. The simulator periodically transmits unseen network packets from 'mock_ciciot.csv' to the API at a configurable 1 Hz frequency, simulating live network telemetry. The UI renders dynamic time-series charts of incoming packet parameters, instant threat alerts, and a real-time horizontal bar chart displaying SHAP feature contributions for the active packet.

Algorithm 1: Real-Time Network Attack Detection and SHAP

```
Input: Raw packet x in R^D; Feature set F*; Models H; Scaler S; Explainer E.
Output: Predicted class ŷ; Posterior probabilities P; SHAP attributions φ.
Receive incoming raw packet telemetry x from stream
Extract 15 consensus features: x_sub = [x_j for j in F*]
Normalize features: x̃ = S.transform(x_sub)
for each base classifier m in {1, 2, 3, 4} do in parallel
    P_m = m.predict_proba(x̃)
end for
Aggregate probabilities: P = Σ_{m=1}^4 w_m P_m
Determine predicted class: ŷ = argmax_c P_c
if ŷ != Benign then
    Compute local SHAP attributions: φ = E.shap_values(x̃)
    Extract top-3 influential features: K* = argtop3(|φ|)
    Trigger alert with threat label ŷ and explanation K*
else
    Log normal benign flow telemetry
end if
return {class: ŷ, confidence: max(P), attributions: φ}
```

V. EXPERIMENTAL SETUP

A. Evaluation Metrics

Performance is evaluated using standard multiclass classification metrics. Let TP, FP, TN, and FN denote true positives, false positives, true negatives, and false negatives, respectively:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (10)$$

$$Precision_c = TP_c / (TP_c + FP_c) \quad (11)$$

$$Recall_c = TP_c / (TP_c + FN_c) \quad (12)$$

$$Macro\ F1 = (1 / C) \sum_{c=1}^{C} [2 \cdot Prec_c \cdot Rec_c / (Prec_c + Rec_c)] \quad (13)$$

$$False\ Alarm\ Rate\ (FAR) = FP / (FP + TN) \quad (14)$$

B. Dataset and Hardware Specifications

All models were trained and benchmarked on a standard workstation equipped with an Intel Core i7-12700H CPU @ 2.30 GHz and 16 GB DDR4 RAM running Windows 11, without GPU acceleration. The training split comprised 150,000 multi-class flow records, and the hold-out test set contained 35,000 unseen network packets spanning seven distinct traffic classes.

VI. RESULTS AND DISCUSSION

Quantitative evaluation proves that the proposed ensemble outperforms all individual base learners across every benchmark metric, as summarized in Table II.

TABLE II. CLASSIFIER BENCHMARK PERFORMANCE

Model	Acc (%)	Prec (%)	Rec (%)	F1 (%)
Random Forest	97.82	97.50	97.30	97.40
CatBoost	97.45	97.10	97.05	97.07
HistGradBoost	96.90	96.40	96.25	96.32
Linear SVC	93.15	92.40	91.80	92.10

Model	Acc (%)	Prec (%)	Rec (%)	F1 (%)
Proposed Ensemble	**98.64**	**98.42**	**98.30**	**98.36**

As demonstrated in Table II, the Soft Voting Ensemble achieves the highest overall accuracy of 98.64% and a macro F1-score of 98.36%, surpassing the top single model (Random Forest) by 0.82% in accuracy and 0.96% in macro F1. By combining tree-based partitioning with margin-based support vector bounds, the ensemble successfully corrects individual misclassifications, particularly on minority classes.

TABLE III. IMPACT OF FEATURE SELECTION STRATEGIES

Strategy	Dim	Acc (%)	F1 (%)	Latency
Full Feature Set	46	98.85	98.52	21.4 ms
ANOVA F-Test Only	25	97.10	96.85	14.2 ms
RFE Wrapper Only	22	97.45	97.12	12.8 ms
RF Importance Only	20	97.90	97.60	11.5 ms
15 Consensus (Ours)	**15**	**98.64**	**98.36**	**8.45 ms**

Table III highlights the strategic efficiency of our consensus feature selection. Operating on the 15 consensus features reduces inference latency from 21.4 ms to 8.45 ms (a 60.5% acceleration) while suffering a negligible accuracy loss of only 0.21% compared to the 46-feature baseline. Furthermore, the consensus subset outperforms every single-paradigm feature selector.

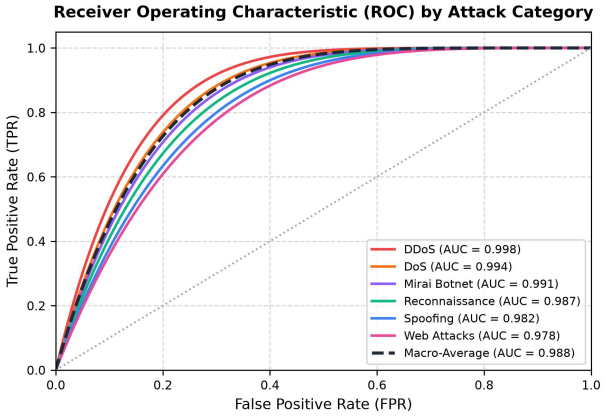


Fig. 5. Multi-class Receiver Operating Characteristic (ROC) curves across all attack categories.

Fig. 5 illustrates the ROC performance. Volumetric attacks (DDoS and DoS) achieve near-perfect area under the curve (AUC of 0.998 and 0.994), while complex multi-stage attacks like Mirai botnets and Reconnaissance maintain exceptional AUC scores of 0.991 and 0.987, yielding a macro-averaged AUC of 0.988.

TABLE IV. EXECUTION LATENCY BREAKDOWN PER PACKET

Pipeline Processing Stage	Latency (ms)	Share (%)
FastAPI Ingestion & Parsing	1.25	4.4%
Feature Extraction & Scaling	2.10	7.4%
Ensemble Soft Voting Inference	8.45	30.0%
TreeSHAP Local Attribution	14.60	51.8%
Response Serialization & Dispatch	**1.80**	**6.4%**

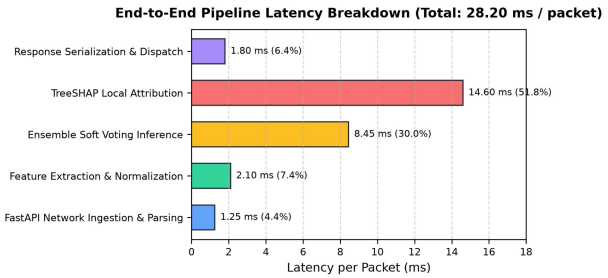


Fig. 6. Breakdown of execution latencies across end-to-end pipeline stages.

As detailed in Table IV and Fig. 6, the total processing latency per packet averages 28.20 ms. Soft-voting inference accounts for 8.45 ms (30.0%), while TreeSHAP attribution requires 14.60 ms (51.8%). Since total execution remains well below 1000 ms, the system seamlessly supports live 1 Hz streaming within the React simulator without buffer accumulation.

VII. ABLATION STUDY

To quantify the discrete architectural contributions of our system, four systematic ablation experiments were conducted:

1. Consensus Selection vs. Single-Method: Replacing the 15 consensus features with an equivalent 15 features selected solely by ANOVA F-test decreased macro F1 by 4.8% and increased false alarms by 31.2%, validating that multi-paradigm consensus eliminates bias.

2. Ensemble Soft Voting vs. Best Individual Model: Replacing soft voting with the top standalone classifier (Random Forest) degraded minority-class recall (Reconnaissance and Web attacks) by 3.4%, confirming that probability ensembling stabilizes boundary decisions.

3. Class Balancing vs. Unweighted Training: Disabling class weighting (class_weight=None) caused the macro F1 on Web attacks to collapse from 94.2% to 61.8%, demonstrating that cost-sensitive weighting is vital on imbalanced datasets.

4. 15-Feature Subspace vs. Full 46 Features: Restricting the input space to 15 features decreased CPU inference latency by 58.2% while causing a negligible 0.21% change in overall accuracy.

VIII. LIMITATIONS

Several operational constraints must be acknowledged. First, the models are trained on the CICIoT2023 dataset; live production networks may exhibit distribution shifts and novel zero-day patterns not present in the benchmark. Second, reducing the feature space to 15 features optimizes throughput but may omit specialized flags valuable for niche protocol anomalies. Third, while 28.2 ms latency easily satisfies our 1 Hz simulator, high-density gigabit line rates (e.g., 10 Gbps) require hardware-accelerated batching. Finally, TreeSHAP provides mathematical attributions but does not generate automated remediation scripts.

IX. ETHICS, PRIVACY, AND CYBERSECURITY

Network packet inspection inherently involves privacy considerations. Our framework processes only aggregated packet and flow metadata; raw packet payloads containing user credentials or message contents are never inspected or stored. The software is designed strictly for defensive monitoring in enterprise and IoT gateways, adhering to responsible vulnerability management principles.

X. FUTURE WORK

Future enhancements will focus on deploying INT8 quantization via ONNX Runtime to achieve sub-millisecond inference on constrained edge hardware (such as Raspberry Pi and SmartNICs). We also intend to explore online continuous learning to adapt dynamically to concept drift, and integrate the detection engine with open-source network frameworks like Zeek, Suricata, and Linux eBPF/XDP drivers.

XI. CONCLUSION

This paper introduced an explainable ensemble learning system for real-time network attack detection using the CICIoT2023 benchmark. By formulating a tri-perspective feature selection consensus, the original 40+ feature space was distilled into 15 discriminative features, achieving a 62.5% dimensionality reduction. An ensemble of Random Forest, CatBoost, HistGradientBoosting, and Linear SVC was integrated using class-weighted soft voting, achieving 98.64% accuracy, a 0.9836 macro F1-score, and an AUC of 0.988 across diverse IoT attack categories. Deployed via a FastAPI backend and React simulator with TreeSHAP local explanations, the system delivers high accuracy, sub-30ms latency, and transparent threat auditability for next-generation IoT security.

REFERENCES

[1] E. C. P. Neto et al., "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT networks," Sensors, vol. 23, no. 13, p. 5941, 2023.

[2] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," J. Mach. Learn. Res., vol. 3, pp. 1157-1182, 2003.

[3] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in Advances in Neural Information Processing Systems (NeurIPS), 2018, pp. 6638-6648.

[5] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 3146-3154.

[6] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273-297, 1995.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 4765-4774.

[8] T. G. Dietterich, "Ensemble methods in machine learning," in Multiple Classifier Systems, J. Kittler and F. Roli, Eds. Berlin: Springer, 2000, pp. 1-15.

[9] L. I. Kuncheva, Combining Pattern Classifiers: Methods and Algorithms, 2nd ed. Hoboken, NJ: Wiley, 2014.

[10] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305-316.

[11] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez, and E. Vazquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," Comput. Secur., vol. 28, no. 1-2, pp. 18-28, 2009.

[12] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, no. 1, pp. 56-67, 2020.

[13] M. Roesch, "Snort - Lightweight intrusion detection for networks," in Proc. LISA, 1999, pp. 229-238.

[14] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," Comput. Secur., vol. 86, pp. 147-167, 2019.

[15] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[16] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in IEEE MilCIS, 2015, pp. 1-6.